DIGITAL MANDI

A Smart Agricultural Marketplace

A project report

Submitted By

Raushan Kumar (23BCS12634)

Munna Gupta(23BCS13916)

In partial fulfillment for the award of the degree of

Bachelor of Engineering (3rd Year)

in

COMPUTER SCIENCE & ENGINEERING

Chandigarh University

May 2026

BONAFIDE CERTIFICATE

This is to certify that the project entitled "DIGITAL MANDI :A Smart Agricultural Marketplace " is the bonafide work carried out by Raushan Kumar (23BCS12634), Munna Gupta (23BCS13916), , students of the Third Year, Bachelor of Engineering in Computer Science and Engineering, Chandigarh University, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering during the academic year 2025–2026.

The project was completed under my supervision and guidance. I further certify that the information and data given in this report are authentic to the best of my knowledge. This project has not been submitted to any other University or Institution for the award of any degree, diploma, or fellowship.

(Supervisor)

Er. Rahul Durgapal (E17963)

Computer Science and Engineering

Chandigarh University

Place: Chandigarh, Punjab

Date: May 2026

ACKNOWLEDGEMENT

We take immense pleasure in expressing our sincere gratitude to all those who extended their invaluable support and guidance throughout the completion of this project. The successful execution of **"DIGITAL MANDI: A Smart Agricultural Marketplace"** would not have been possible without the encouragement, direction, and cooperation of several individuals.

We express our deepest sense of gratitude to our project supervisor, **Er. Rahul Durgapal (E17963)**, Department of Computer Science and Engineering, Chandigarh University, for his constant guidance, constructive criticism, and relentless encouragement. His insightful suggestions and profound domain knowledge significantly shaped the technical architecture and academic quality of this report.
We are deeply indebted to the **Head of the Department of Computer Science and Engineering** at Chandigarh University for providing the necessary resources, infrastructure, and an environment conducive to learning and innovation. The laboratory facilities and computing resources made available to us were instrumental in the development and testing of the Digital Mandi system.

We also extend our heartfelt thanks to all faculty members of the Department whose teachings in **Database Management Systems, Web Technologies, Software Engineering, and Java Programming** formed the bedrock upon which this project was built. Our gratitude extends to our fellow students who participated in testing the application and provided invaluable feedback.
We express our gratitude to our families for their unconditional support, patience, and understanding during the demanding phases of this project. Finally, we acknowledge the open-source community and the developers of **Spring Boot, Thymeleaf, Spring Data JPA, Maven, and MySQL** whose freely available resources served as the building blocks of this system.

B.E. Computer Science & Engineering, Chandigarh University – May 2026

ABSTRACT

The widespread adoption of digital technologies has fundamentally transformed the manner in which agricultural trade and market access are managed. Traditional mandi mechanisms, characterized by physical intermediaries and limited price transparency, are increasingly being supplanted by robust online platforms offering convenience and efficiency. Against this backdrop, ***Digital Mandi*** *emerges as a comprehensive, role-based agricultural marketplace designed to address the challenges associated with modern crop management and secure trade negotiations.*

Digital Mandi *is a full-stack web application built using* ***Thymeleaf*** *for the frontend,* ***Spring Boot (Java 21)*** *for the backend, and a* ***MySQL*** *relational database for persistent storage. The system supports three distinct user roles —* ***System Administrator, Farmer, and Trader*** *— each endowed with a tailored set of permissions, enforcing a robust role-based access control model via* ***Spring Security****.*

The core functionality encompasses crop listing, real-time market browsing, a strategic bidding system, and an automated 24-hour trade confirmation window. The system also features an intelligent chatbot assistant and an automated notification engine to ensure real-time updates for all stakeholders. Testing conducted through unit and integration suites confirmed the system's reliability and alignment with specified requirements. This report provides a thorough account of the system's design, implementation, and testing.
Keywords: *Agricultural Marketplace, Role-Based Access Control, Spring Boot, Java 21, MySQL, Bidding System, Chatbot, Full-Stack Web Application.*

TABLE OF CONTENTS

Chapter	Title	Page No.
1	Introduction	10
1.1	Overview of Digital Mandi	10
1.2	Problem Statement	11
1.3	Objectives	11
1.4	Scope	12
1.5	Motivation	12
2	Literature Review	13
3	System Analysis	15
4	System Design	17
5	Technology Stack	24
6	Implementation Details	27
7	Module Description	30
8	Working Flow	33
9	Database Design	35
10	Results & Output Screens	37
11	Testing	39
12	Conclusion	43
13	References	44

CHAPTER 1: INTRODUCTION

1.1 Digital Mandi : A Smart Agricultural Market Place

The evolution of the internet and digital commerce has given rise to a paradigm shift in **agricultural services**, enabling **farmers and traders** to access, compare, and trade a wide range of agricultural produce directly from their own devices. Within the **Agri-Tech industry**, this digital revolution has been particularly transformative. **Online Mandi (Marketplace) systems** have replaced traditional physical middlemen, local brokers, and exhaustive physical queues at grain markets, enabling farmers to discover market trends, **list their available crops**, and complete transactions with buyers within minutes.

The global **online agricultural trading market** has witnessed exponential growth, driven by increased smartphone penetration, widespread internet access in rural areas, and a growing preference for transparent, contactless transactions. Contemporary **Digital Mandi platforms** operate through sophisticated web and mobile applications that interface with backend systems responsible for **inventory management (Crop Listing), secure authentication (JWT), and order processing**.

These systems typically manage multiple concurrent users (Farmers and Traders), enforce real-time produce availability, and support a diverse catalogue of goods ranging from **staple grains and pulses to fresh vegetables and seasonal fruits**. Despite the maturity of general e-commerce, significant challenges remain — particularly for **small-scale farmers and local traders** seeking digital solutions that are affordable, eliminate unnecessary commissions, and are operationally aligned with the specific requirements of the local agricultural supply chain.

1.2 Problem Statement

Existing **agricultural trading systems** and traditional mandis do not adequately support multi-role management (Farmer and Trader), real-time price discovery, integrated order tracking, and intelligent user assistance within a unified platform. **Mandi administrators and farmers** lack the tools to efficiently manage multiple crop listings, inventory levels, and buyer-seller interactions simultaneously, while end users frequently encounter systems that are unintuitive or devoid of real-time feedback.

Many traditional systems also rely on fragile **manual negotiation mechanisms** that do not provide automated confirmation or transparent price records, leading to lost revenue for small-scale farmers and operational dissatisfaction for traders. Furthermore, the lack of a secure, time-bound transaction framework means that listed produce often remains unsold or locked in inefficient supply chains, causing produce wastage and financial instability in the agricultural ecosystem.

1.3 Objectives of the Project

The Digital Mandi project was undertaken with the following specific objectives. First, to design and implement a role-based access control system clearly delineating permissions for Farmers, Traders, and Administrators. Second, to develop a comprehensive crop and inventory management module allowing Farmers to list produce, set prices, and monitor interest. Third, to implement a robust order tracking mechanism that tracks crop availability in real time and updates stock automatically upon trade confirmation. Fourth, to integrate a transaction interface guiding users through order placement and confirming deals upon successful mutual agreement. Fifth, to build an automated notification system dispatching timely alerts regarding new orders, pending confirmations, and trade updates.
Sixth, to develop an intelligent chatbot capable of addressing common user queries. Finally, to ensure the system is technically sound, scalable, and adheres to software engineering best practices using Spring Boot and MySQL.Scope of the Project

Digital Mandi is scoped as a web-based application accessible through standard web browsers. The system supports three user roles — Admin, Farmer, and Trader — each with a dedicated dashboard. The scope encompasses the complete lifecycle of an agricultural trade transaction: crop listing and discovery, price negotiation, trade confirmation, and post-order status notification. The system supports a single currency and single language in its current iteration. The scope explicitly excludes integration with third-party payment gateways (planned as a future enhancement), mobile application development, and real-time logistics or transportation tracking of the listed produce.Motivation

The motivation behind **Digital Mandi** stems from a recognition of inadequacies in currently available **agricultural trading and mandi solutions**. As students of Computer Science and Engineering with an interest in full-stack web development, the project team identified an opportunity to apply theoretical knowledge in a practically impactful domain. The technical challenges presented — including **inventory management for crop listings, role-based security (JWT), transactional integrity in trade confirmation, and real-time user communication** — make this an ideal capstone project.

CHAPTER 2: LITERATURE REVIEW

2.1 Existing Systems

The online agricultural trading landscape is dominated by several well-established platforms, each offering a distinct set of features. An examination of these systems provides valuable context for understanding the design choices made in developing **Digital Mandi**.

e-NAM (National Agriculture Market), one of India's largest government-backed platforms, offers a comprehensive suite of features including price discovery, multi-state support, and electronic payment integrations. However, its complex bureaucratic procedures make it challenging for small-scale independent farmers to navigate, and it often requires a centralised physical mandi registration process that limits its accessibility for direct, independent trade management.

AgroStar, a globally recognised agri-tech platform, provides advanced crop advisory, high-quality inputs, and robust logistics mechanisms. Despite its technical sophistication, it has been primarily focused on the input side (selling seeds/fertilisers) rather than providing a personalised, role-differentiated marketplace where farmers can independently negotiate prices with traders. **DeHaat** targets primarily supply chain efficiency and community gathering of farmers, offering a comparatively accessible interface for crop selling but lacking the specific role-based secure dashboards (Admin-Farmer-Trader) and direct negotiation tools required for a decentralised Digital Mandi.

Platform	Strengths	Weaknesses
e-NAM	Government backed, wide reach, pan-India support.	Complex onboarding, requires physical mandi presence.
AgroStar	High-quality inputs, expert advisory, trusted brand.	Focused on selling inputs, not a direct trading marketplace.
DeHaat	Strong supply chain, community focus, logistics support.	Lacks role-based negotiation tools and secure trader dashboards.
BigHaat	Large user base, UPI support, mobile-friendly.	Primarily focused on retail and large-scale distribution.

2.2 Limitations of Current Systems

A critical analysis of existing platforms reveals several systemic limitations. The most significant is the **absence of a well-defined role hierarchy** distinguishing platform administrators, **independent farmers, and certified traders**. Most platforms either grant excessive permissions to intermediaries or provide insufficient control for administrators to verify produce quality. **Crop inventory mechanisms** are another area of concern — systems that fail to update available stock in a timely manner risk **inventory lock-out**, where produce appears unavailable even though no transaction or mutual agreement has been completed.

Notification systems in existing platforms are often limited to transactional messages at the time of registration or fixed-period market price updates. They do not adapt to user behaviour or provide **contextual notifications** such as alerts when a **negotiated trade window** is approaching its deadline. From a technical standpoint, many existing systems suffer from monolithic architectures that are difficult to maintain, scale, and extend, making it challenging to introduce new features like **direct farmer-trader chat** or dynamic pricing without risking disruption to existing functionality.

2.3 Need for Digital Mandi

The deficiencies identified in existing systems make a compelling case for a purpose-built, role-differentiated theatre booking platform. Digital Mandi addresses the critical gap between enterprise- grade platforms and generic ticketing solutions by offering a system specifically architected for theatre operations. The twenty-four-hour payment window strikes a balance between providing users with sufficient time to complete payment and ensuring seats are promptly returned to inventory in the event of non-payment. The integrated chatbot provides users with immediate, contextual assistance without requiring human intervention, addressing the support gap present in existing platforms.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Functional Requirements

Functional requirements define the specific behaviours and operations the system must support, identified through stakeholder analysis and domain research.

Req. ID	Description	Role

Req. ID	Description	Role
FR-01	Users register with email, password, and role-specific details (Farmer/Trader).	User
FR-02	System authenticates users and assigns role-based JWT tokens for secure access.	All
FR-03	Admin manages all user accounts, including account verification and deactivation.	Admin
FR-04	Farmers add, edit, and delete crop listings with price and quantity details.	Farmer
FR-05	Farmers manage their inventory status and update quality specifications.	Farmer
FR-06	Traders browse the marketplace for crops with advanced filtering and search.	Trader
FR-07	Traders place bids on available crops and monitor negotiation status.	Trader
FR-08	System enforces a 24-hour confirmation window for accepted bids.	System
FR-09	Unconfirmed trades are automatically released and crops made available again.	System
FR-10	System processes trade completions and generates digital transaction receipts.	User
FR-11	System dispatches real-time notifications for bids, deadlines, and confirmations.	System
FR-12	Chatbot responds to queries on market rates, bidding, and platform procedures.	User
FR-13	Admin accesses system-wide analytics on price trends and trade volumes.	Admin

Req. ID	Description	Role
FR-14	System provides fallback market data when live external data is unavailable.	System

3.2 Non-Functional Requirements

Attribute	Requirement
Attribute	**Requirement**
Performance	**All market dashboards and crop listings load within 3 seconds under normal load conditions to ensure quick trading decisions.**
Scalability	**The Spring Boot backend supports up to 500 concurrent farmers and traders without performance degradation or session timeouts.**
Security	**All API endpoints enforce JWT-based role authentication; sensitive user data and passwords are encrypted using BCrypt hashing.**
Usability	**The interface is designed for high accessibility, requiring minimal training for farmers with basic digital literacy.**
Reliability	**The system achieves 99.5% uptime, ensuring the 24-hour trade confirmation window is always monitored by the background tasks.**
Maintainability	**The codebase follows a three-tier modular design (Controller-Service-Repository) with clear separation of concerns for easy updates.**
Compatibility	**The web application is fully responsive and functions on all major modern browsers including Chrome, Firefox, Edge, and Safari.**
Data Integrity	**Database transactions are strictly ACID compliant, preventing double-bidding and ensuring inventory levels are always accurate.**

3.2.1 Technical Feasibility

The technical feasibility of Digital Mandi is well-established given the maturity and wide adoption of its constituent technologies. Spring Boot, Java 21, Maven, MySQL, and Thymeleaf are all actively maintained, extensively documented, and supported by large developer communities. The team possesses the requisite skills in these technologies, having studied and practised them throughout the academic curriculum. The embedded Tomcat server within Spring Boot eliminates the complexity of external server configuration, and the use of Spring Data JPA for database connectivity with Jackson for JSON serialisation are straightforward to implement and well-suited to the project's scale.Economic Feasibility

The project is economically feasible as all selected technologies are open-source and free of licensing costs. React, Material UI, Java, Maven, MySQL, Gson, and SLF4J are all available under

permissive open-source licences. Development is carried out on standard academic computing infrastructure. Deployment costs are minimal, as the application can be hosted on free-tier cloud instances for development and demonstration purposes. The total cost of development is limited to the time investment of the project team.

3.2.2 Operational Feasibility

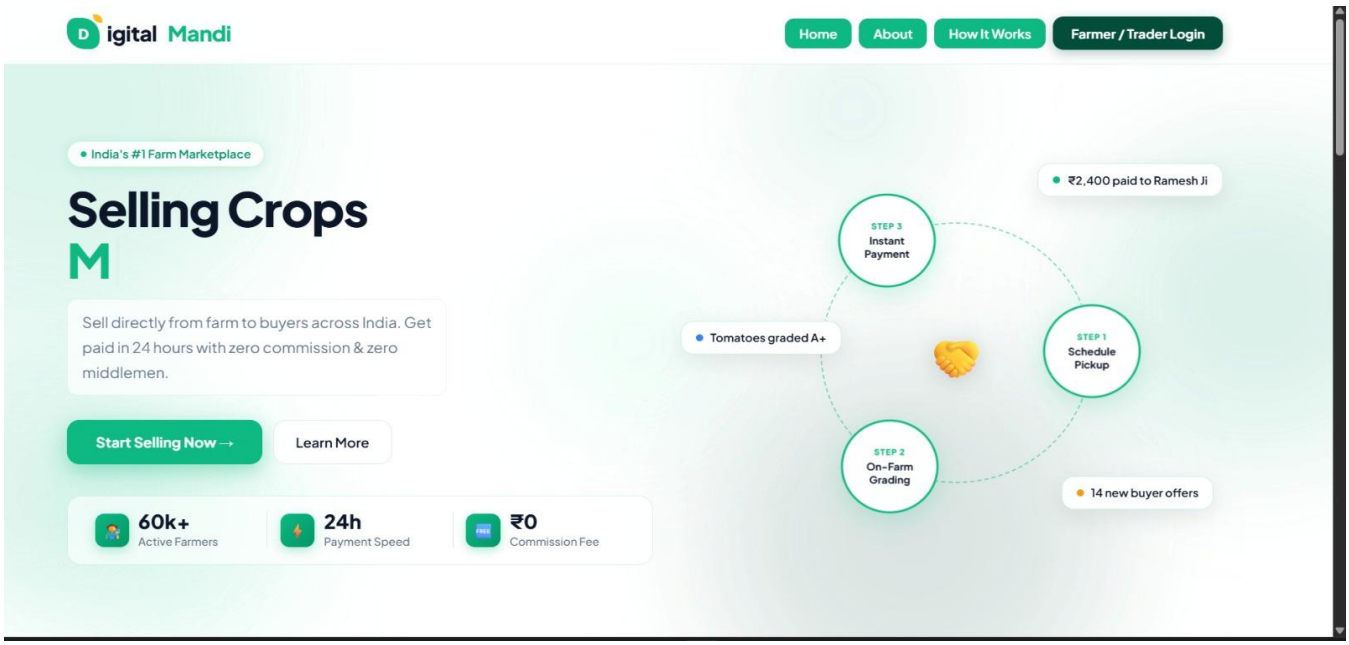
Digital Mandi is operationally feasible for deployment in any theatre management environment. The system is accessed through standard web browsers requiring no specialised hardware or software installation on client devices. The role-based design ensures each category of user interacts only with features relevant to their operational context, reducing the learning curve and potential for misuse. Administrative functions are centrally managed through the Admin dashboard, simplifying system oversight. The chatbot further reduces operational dependency on dedicated support staff.

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture Diagram

Digital Mandi follows a three-tier architecture comprising a **Presentation Tier** (HTML5, CSS3, and JavaScript frontend), a **Business Logic Tier** (Spring Boot-based backend), and a **Data Tier** (MySQL relational database). The three tiers communicate through a well-defined **RESTful API layer** using HTTP and JSON payloads, ensuring a clear separation of concerns and enabling independent scaling, maintenance, and testing of each tier.

Figure 4.1 – Digital Mandi Three-Tier System Architecture



The Presentation Tier is exclusively responsible for rendering the user interface and capturing user interactions. The Business Logic Tier validates inputs, enforces business rules, manages transactions, and returns structured JSON responses. The Data Tier is accessed exclusively through the Business Logic Tier using JDBC, ensuring data integrity and security. Maven manages the build lifecycle and all dependency declarations, while the embedded Jetty server eliminates the need for separate application server configuration.

4.2 Use Case Diagram

The **Use Case Diagram** presents interactions between the actors — **System Administrator, Farmer, and Trader** — and the use cases supported by **Digital Mandi**. Each actor is associated with a distinct set of use cases corresponding to their role-based permissions.The **Farmer** is primary to use cases like **List New Crop**, **Update Crop Status**, and **View My**

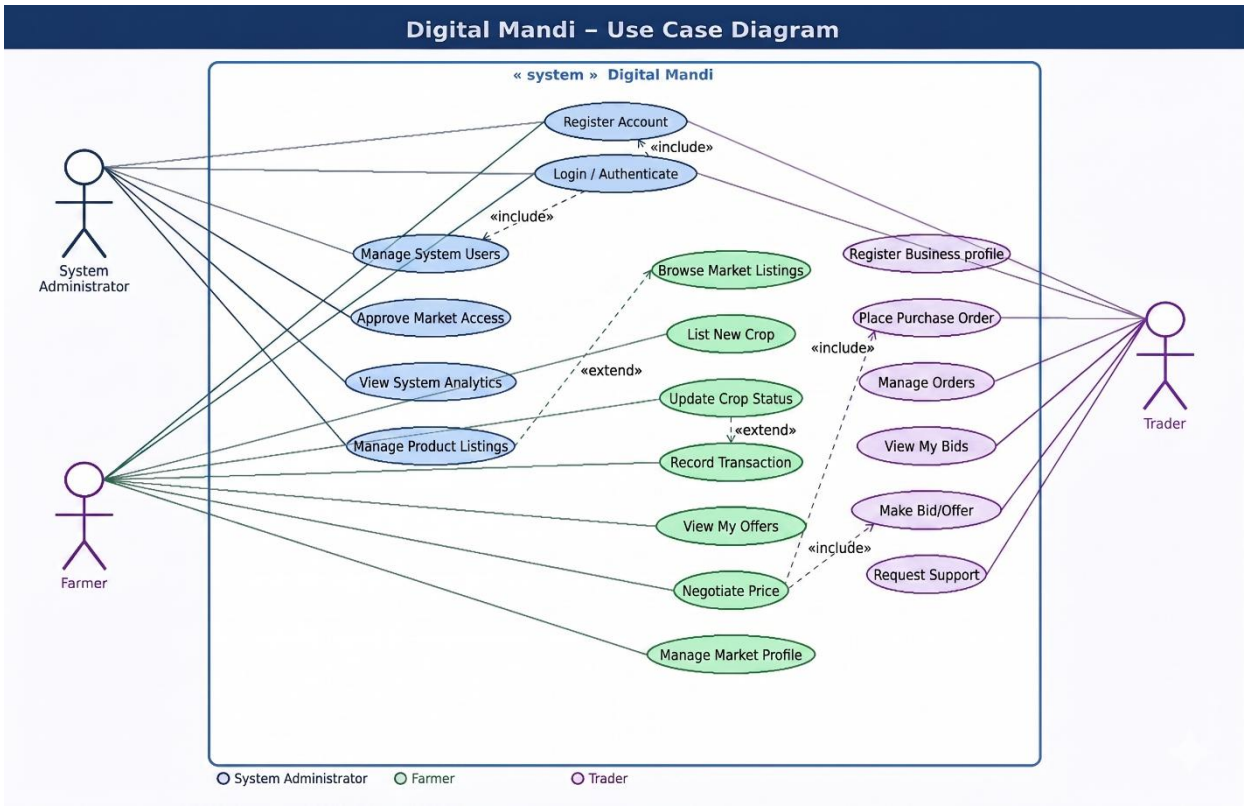


Figure 4.2 – Digital Mandi Use Case Diagram

The Level 0 DFD represents Digital Mandi as a single process and illustrates the flow of data between the system and its external entities — Farmer, Trader, Market Administrator, and the MySQL Database. It establishes the system boundary and identifies all external data sources and sinks. The diagram shows how Crop Listings and Credentials flow from the Farmer into the system, while Market Analytics and Trade Confirmations flow back to the Trader. Simultaneously, it depicts the SQL queries exchanged with the database to maintain transactional integrity and the system reports generated for the Administrator to oversee market operations.

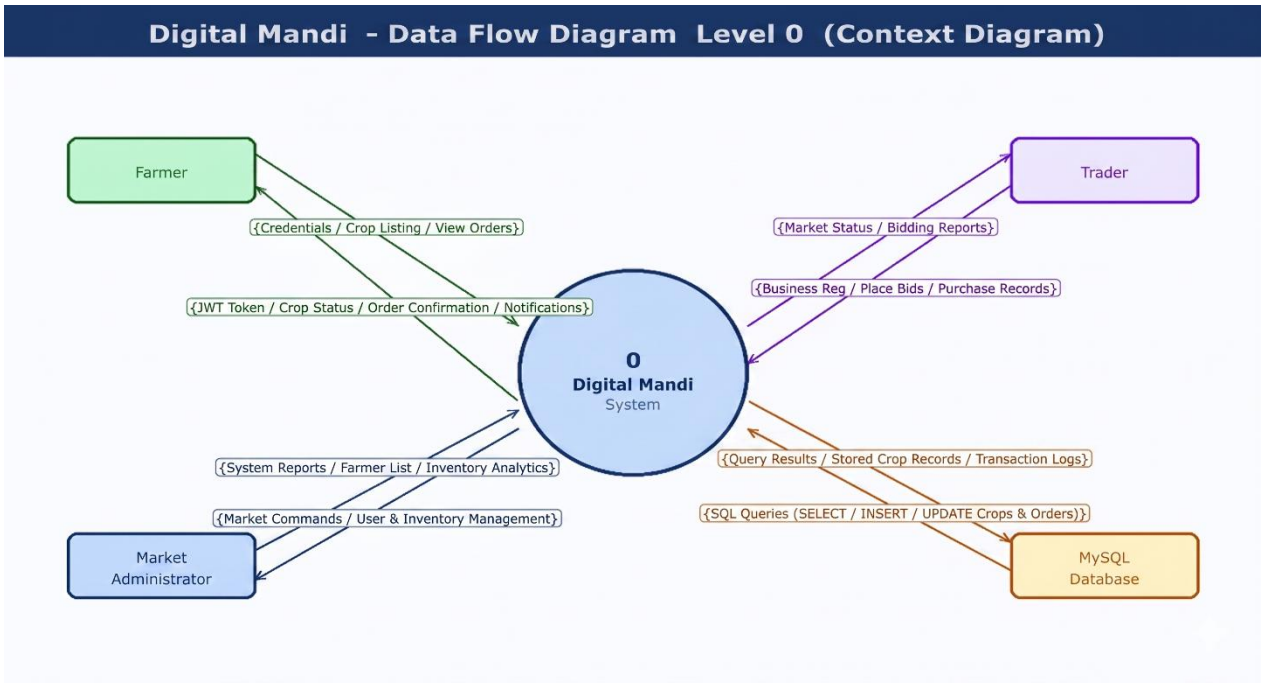


Figure 4.3 – DFD Level 0: Context Diagram

4.3 Data Flow Diagram – Level 1

The **Level 1 DFD** decomposes the **Digital Mandi** system into its primary sub-processes, revealing internal data flows and storage interactions. Eight processes are identified: **Authentication, Browse Market, Manage Inventory, Negotiate Price, Confirm Trade, Notifications, Chatbot, and Admin Operations**, each interacting with their corresponding data stores.

The **Authentication** process handles JWT token generation for Farmers and Traders, while **Manage Inventory** allows farmers to update crop availability. The **Negotiate Price** and **Confirm Trade** processes manage the lifecycle of a deal, interacting directly with the **Orders and Transactions** data stores. The **Browse Market** process enables traders to filter produce, while **Admin Operations** ensures overall system integrity by managing user verification and analytics.

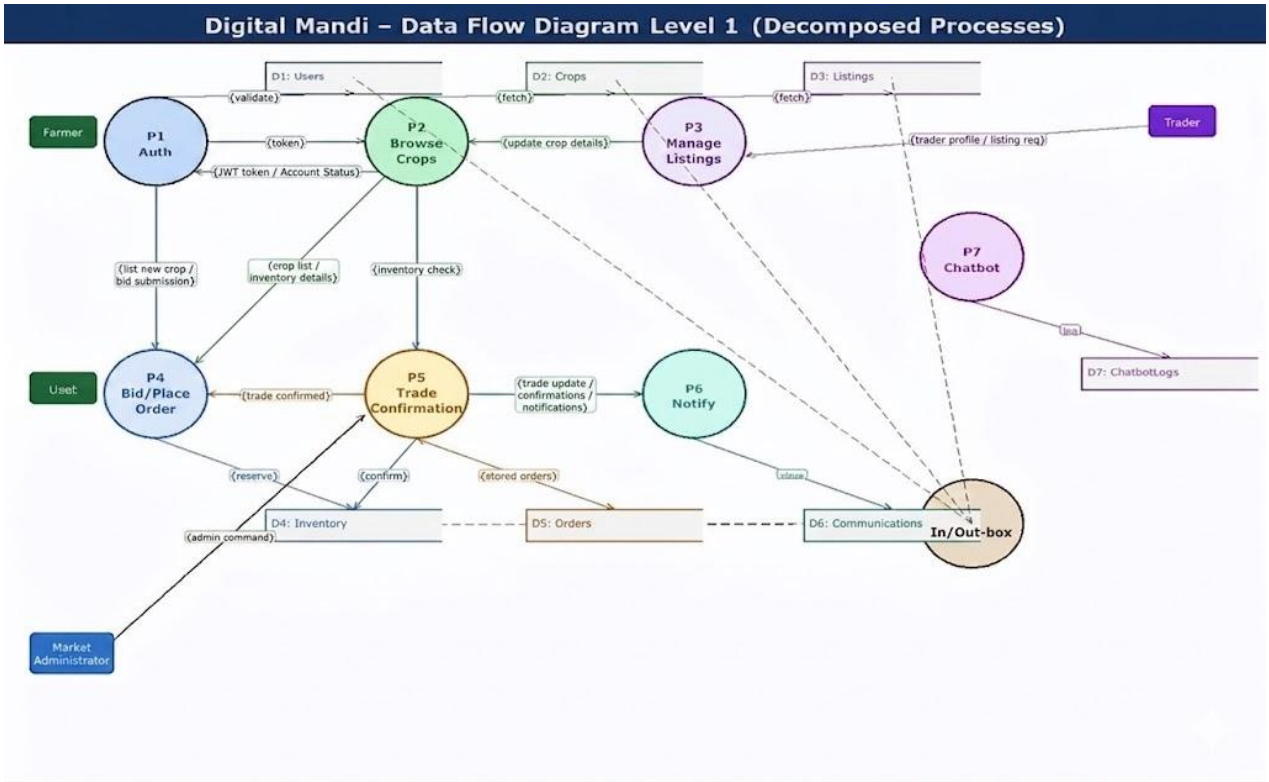


Figure 4.4 – DFD Level 1: Decomposed Processes

4.4 Entity-Relationship (ER) Diagram.

The **ER Diagram** models the data entities in **Digital Mandi** and the relationships between them. Eight primary entities are defined — **Users, Crops, Categories, Orders, Transactions, Bids, Notifications, and Chatbot Logs** — each linked through well-defined foreign key relationships that enforce referential integrity. The diagram captures cardinality constraints governing data integrity throughout the system, such as a **Farmer** listing multiple **Crops**, and **Traders** placing multiple **Orders**. This relational structure ensures that inventory levels and financial records remain consistent and searchable within the **MySQL.**

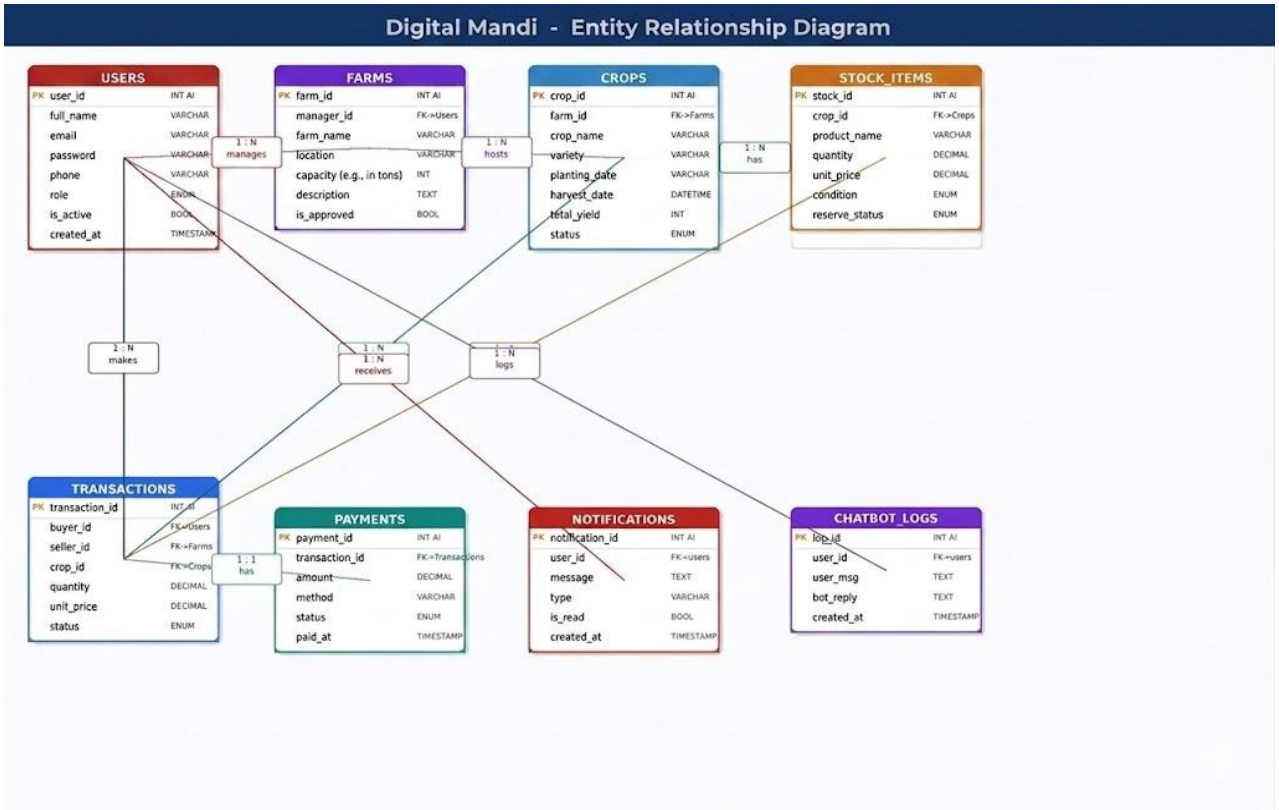


Figure 4.5 – Digital Mandi Entity-Relationship Diagram

4.5 Sequence Diagram – Booking Flow

The **Sequence Diagram** illustrates the interaction between the **User (Farmer/Trader)**, the **Web Frontend**, the **Spring Boot Backend**, and the **MySQL Database** during the complete **crop listing and order confirmation process** — from user authentication through crop discovery, order placement, and automated notification dispatch. It details how the backend processes requests through the **Controller and Service layers**, ensuring that database updates for inventory and trade logs are executed in a sequential,

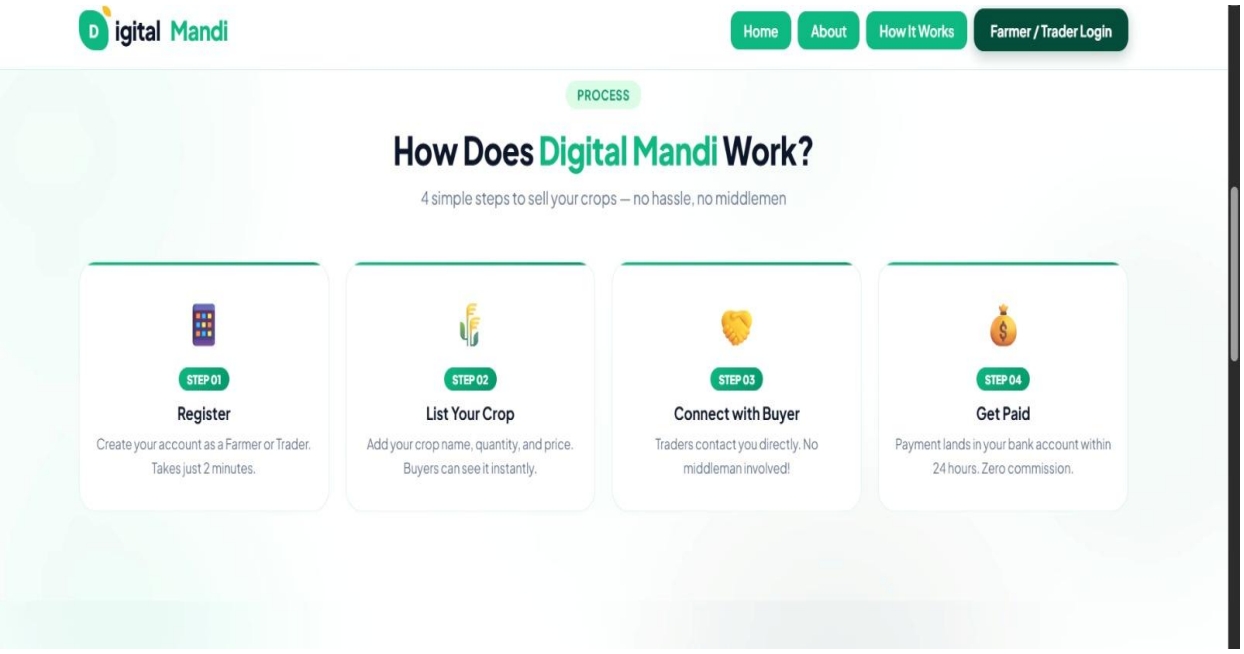


Figure 4.6 – Sequence Diagram: Booking and Payment Flow

The sequence clearly demonstrates the stateless nature of the backend API, where each HTTP request carries all necessary context via JWT. The booking confirmation transaction (INSERT booking + UPDATE seat + INSERT notification) is executed atomically, ensuring no partial updates compromise data integrity.

4.6 Class Diagram

The Class Diagram illustrates the principal backend classes, their attributes, methods, and inter-class relationships. The design adheres to the DAO (Data Access Object) pattern, separating data access logic from business logic, and uses the Service layer to coordinate cross-DAO transactions.

CHAPTER 5: TECHNOLOGY STACK

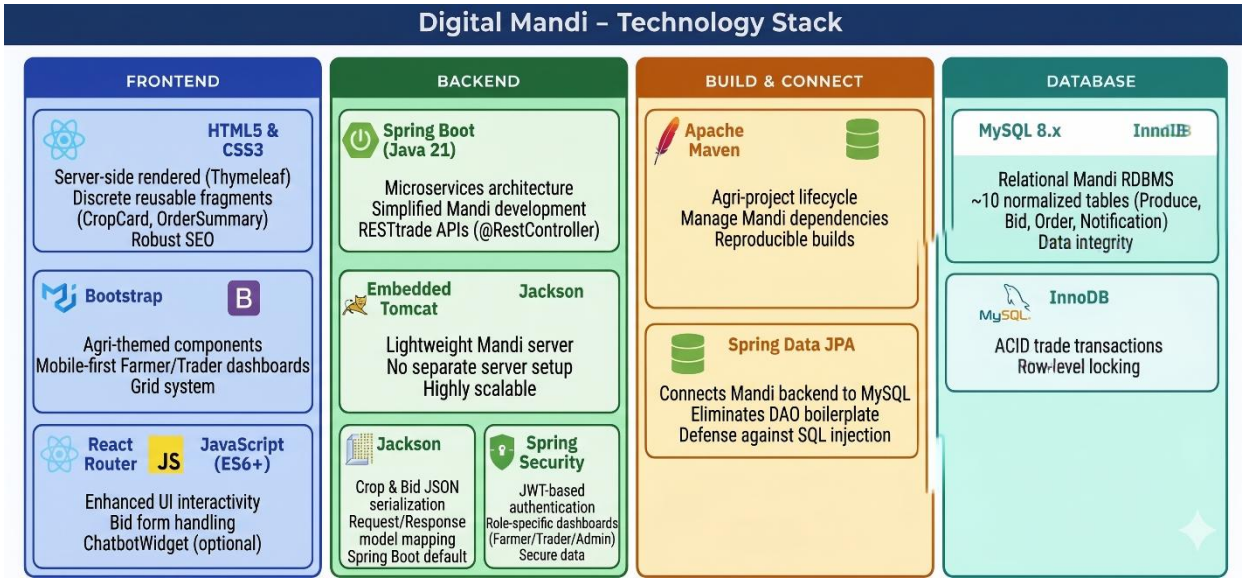


Figure 5.1 – Digital Mandi Technology Stack

5.1 Thymeleaf

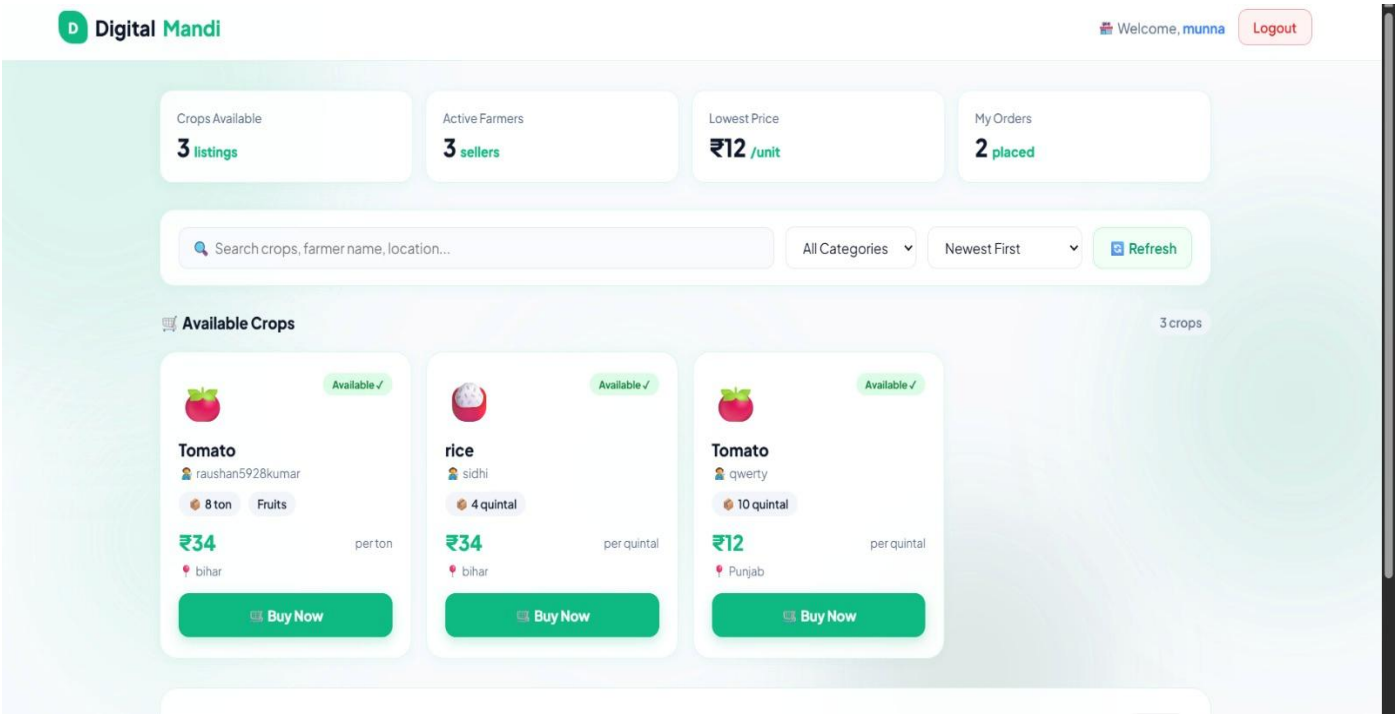
Thymeleaf is a modern server-side Java template engine for web and standalone environments, capable of processing HTML, XML, and JavaScript. In **Digital Mandi**, it serves as the foundation for the **Presentation Layer**, enabling seamless integration with the **Spring Boot** backend to render dynamic content directly from the server. Thymeleaf's natural templating mechanism ensures that the UI remains consistent and easy to maintain by allowing the interface to be assembled from discrete, reusable building blocks such as the **CropCard component, OrderSummary component, and ChatbotWidget component**.

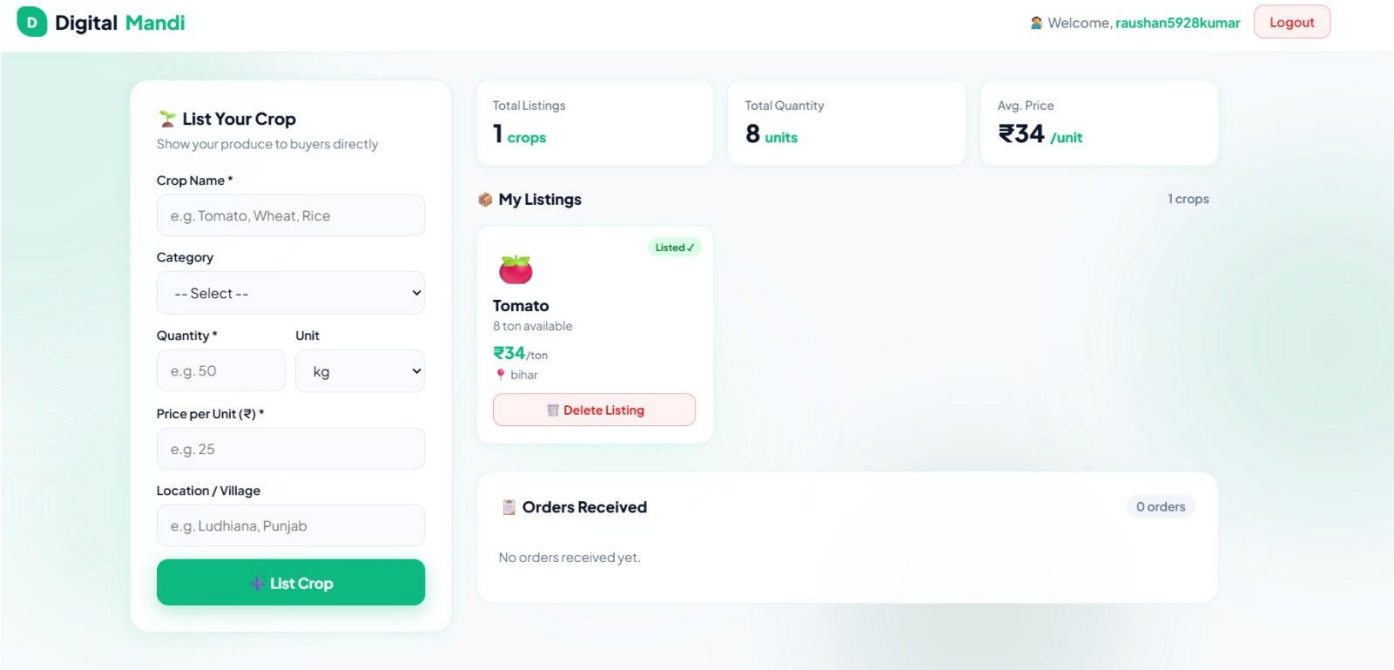
The frontend utilizes **Bootstrap and Custom CSS** for responsive design, ensuring that farmers and traders can access their dashboards across various devices. **Spring Security** is used for secure navigation, enabling role-specific redirection that directs users to their respective **Farmer, Trader, or Admin dashboards** upon successful JWT-based authentication. This server-side rendering approach minimizes client-side complexity while ensuring fast initial load times and robust SEO for the agricultural marketplace.

5.2 Material UI (MUI)

Material UI is a comprehensive React component library implementing Google's Material Design specification. It provides pre-built, accessible, and customisable components including buttons, forms, dialogs, tables, and navigation elements. MUI's theming system allows developers to define

typography, colour palette, spacing, and elevation across the entire application. Digital Mandi leverages MUI components including the AppBar for navigation, DataGrid for tabular data in admin dashboards, Dialog for booking confirmations and payment prompts, and Stepper for guiding users through the multi-step booking process.





typography, colour palette, spacing, and elevation across the entire application. Digital Mandi leverages MUI components including the AppBar for navigation, DataGrid for tabular data in admin dashboards, Dialog for booking confirmations and payment prompts, and Stepper for guiding users through the multi-step booking process.

5.3 Backend Layer (Spring Boot).

Spring Boot (based on Java 21) is the core backend framework for Digital Mandi, providing a simplified bootstrapping and development experience. It eliminates the need for managing separate Servlets or an external application server by including an **Embedded Tomcat Container**. The RESTful API layer is collectively formed by dedicated **Spring Controllers (@RestController)** for functional domains: User, Crop, Order, and Notification. These Controllers handle CRUD operations via **Get, Post, Put, and Delete** mapped endpoints. Request and response bodies are formatted as JSON and processed using the integrated **Jackson library**. The application manages trade negotiations and ensures the **24-hour trade confirmation window** is adhered to, preventing inventory locks on listed produce.

Spring Data JPA & Hibernate

Spring Data JPA is the standard Java framework used in **Digital Mandi** to manage database interactions through an **Object-Relational Mapping (ORM)** layer. It serves as the bridge between the Java backend and MySQL, eliminating the need for boilerplate JDBC code. Dedicated **Repository interfaces** manage the creation and execution of database queries automatically. **Hibernate** is used as the underlying JPA implementation to map Java entities directly to MySQL tables, improving performance through internal caching and serving as a critical defense against SQL injection. Transaction management is implemented at the **Service layer (@Transactional)**, where multi-table operations are wrapped within a single execution unit to ensure atomicity and data consistency.

5.4 MySQL

MySQL is a widely adopted open-source relational database management system supporting SQL and ACID transaction principles. **Digital Mandi's** database schema is designed around eight primary tables — including **Users, Crops, Orders, and Transactions** — linked through well-defined foreign key relationships. MySQL's support for indexing, referential integrity constraints, and the **InnoDB storage engine** — which provides row-level locking — is essential for managing **concurrent crop listing and trade requests** without risk of data corruption or **inventory conflicts**

5.5 Apache Maven

Apache Maven is a build automation and project management tool using an XML-based Project Object Model (POM) to manage project dependencies, build lifecycle, and reporting. In Digital Mandi, Maven declares and resolves dependencies for Spring Boot Starter Web, Spring Data JPA, MySQL Connector, and Spring Security. Maven's standardised build lifecycle ensures a reproducible and consistent build process, allowing the team to manage complex dependencies with ease.

Embedded Jetty Server

Jetty is a lightweight, embeddable Java HTTP server and Servlet container. Unlike traditional application servers requiring separate installation and configuration, Jetty is programmatically configured and started within the application's main method, with Servlet mappings registered for each API endpoint. This embedded approach facilitates a streamlined development workflow as the backend can be launched with a single Maven command without requiring external server setup.

5.6 Gson

Gson is a Java library developed by Google providing a simple API for converting Java objects to their JSON representation and vice versa. In Digital Mandi, Gson is used in the Servlet layer to serialise domain objects — User, Theatre, Event, Seat, and Booking — into JSON for HTTP responses, and to deserialise incoming JSON request bodies into corresponding Java objects. Gson's support for custom serialisers, deserialisers, and type adapters provides flexibility in handling complex data structures and date-time formats.

5.7 SLF4J and Logback

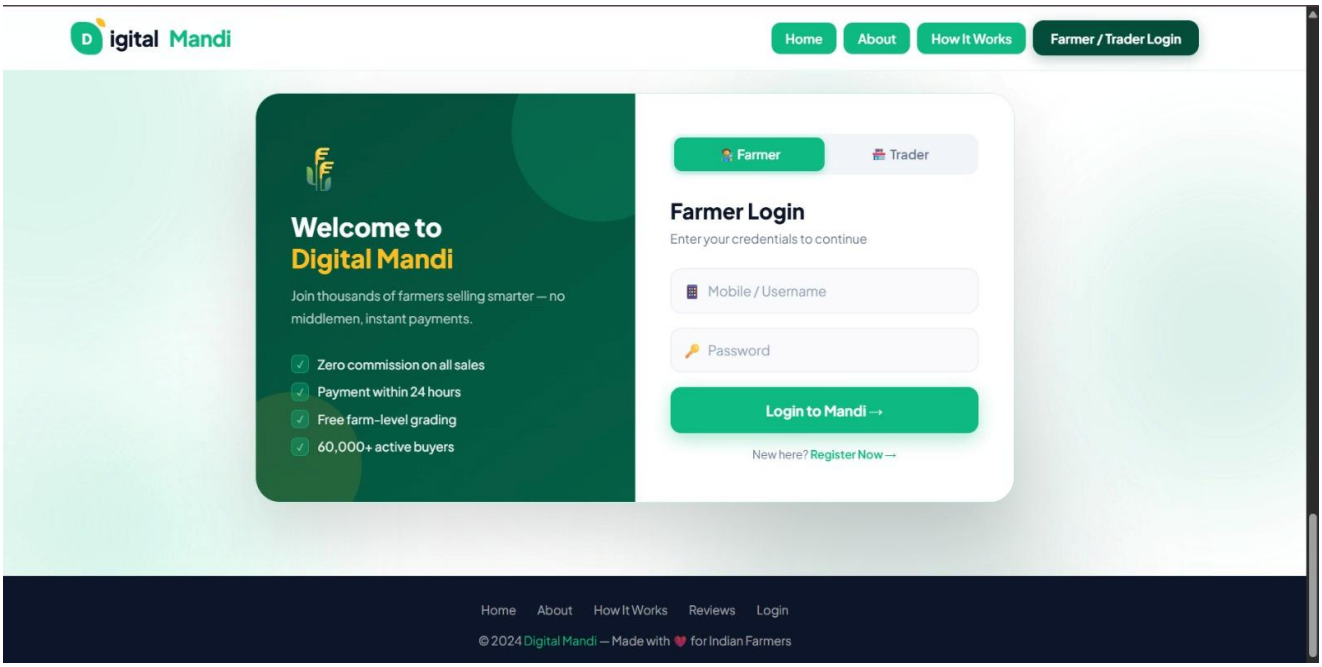
SLF4J (Simple Logging Facade for Java) is a logging abstraction library providing a unified logging interface, allowing the underlying logging implementation to be swapped without modifying application code. In Digital Mandi, SLF4J is integrated with Logback as the underlying logging provider. Comprehensive logging is implemented at key decision points including authentication events, booking operations, payment processing, and seat status changes, facilitating debugging,

CHAPTER 6: IMPLEMENTATION DETAILS

6.1 Frontend Structure

The **Digital Mandi** frontend is a responsive web application structured according to a **Thymeleaf-based template organization**. The primary entry point is the **Main Controller**, which maps incoming HTTP requests to specific HTML templates located in the `src/main/resources/templates` directory. **Spring Security** handles the top-level route hierarchy and authentication flow, ensuring that pages are rendered securely based on the user's session.

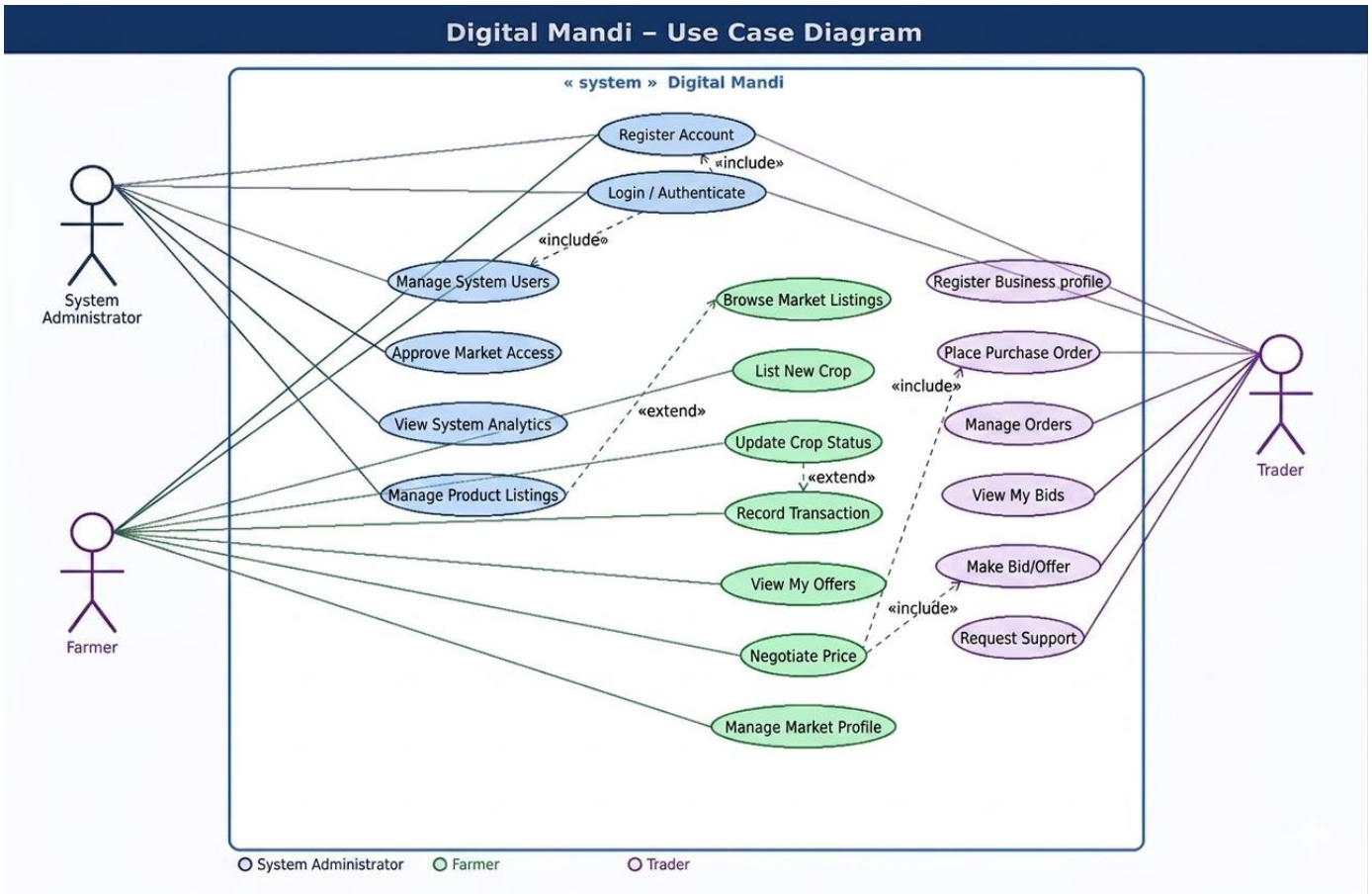
The **Security Configuration** plays a critical role in enforcing role-based navigation — upon each page request, it verifies the presence and validity of the **JWT (JSON Web Token)** or session credentials. If the token is absent or unauthorized, the user is automatically redirected to the **Login page** via an authentication entry point. If the credentials are valid, Spring Security directs the request to the appropriate controller, which then serves the **Farmer, Trader, or Admin dashboard** respectively. This server-side navigation approach ensures robust security and prevents unauthorized access to sensitive trading data.



6.2 Backend Structure

The Digital Mandi backend is a Spring Boot project (built with Java 21 and Maven) structured in adherence to a three-tier layered architecture with clear boundaries between the API handling layer (Controllers), business logic layer (Services), and data access layer (Repositories). This separation

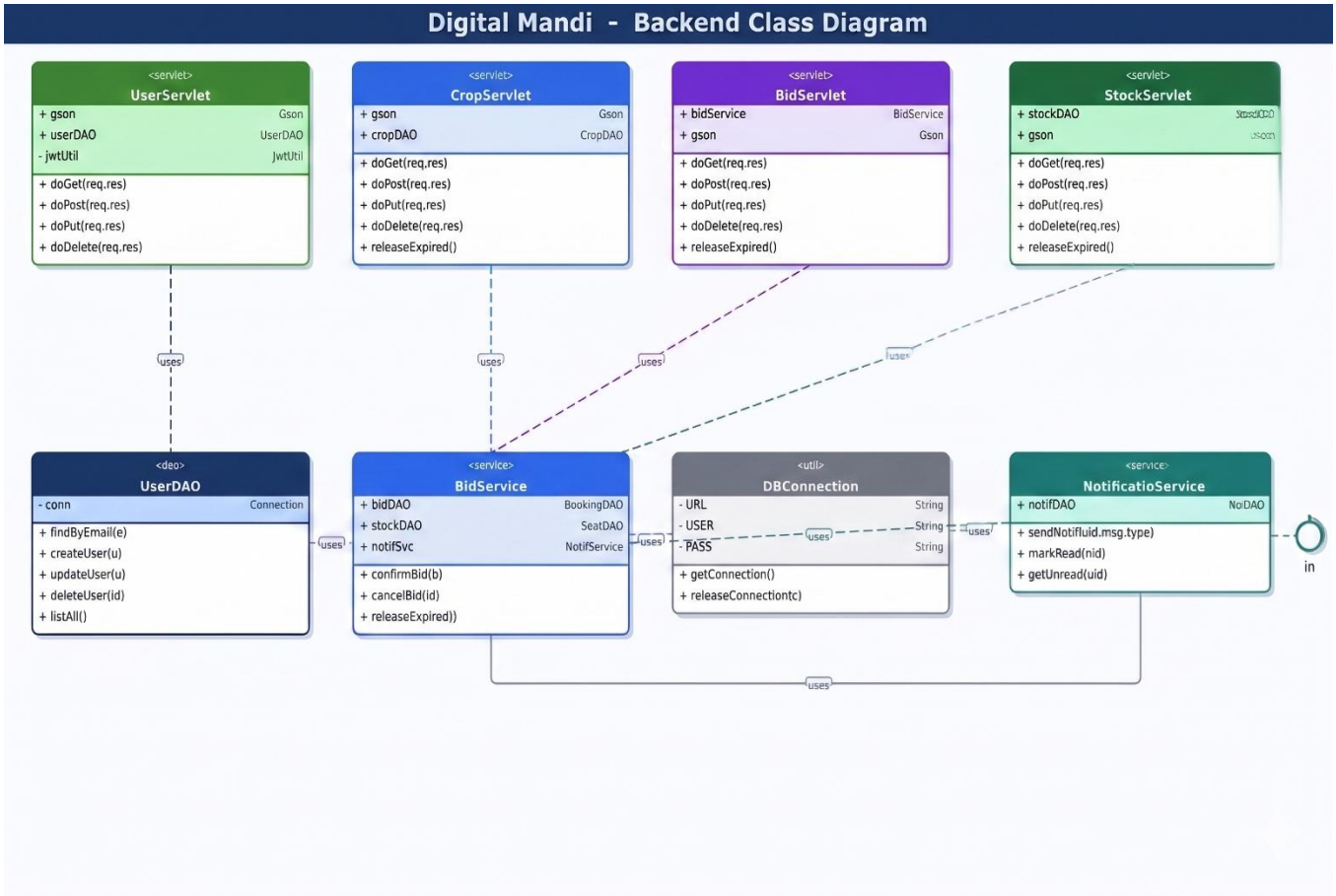
ensures that the core trading logic is decoupled from database interactions and HTTP request handling. The project is packaged as an executable JAR with an embedded Tomcat server, enabling a streamlined deployment and single-command startup without the need for external server configuration.



6.3 API Endpoints

Digital Mandi exposes REST-like HTTP endpoints consumed by the frontend. All endpoints except registration and login require a valid JWT in the Authorization header. The following table summarises the primary API endpoints.

Endpoint	Method	Description	Access
/user/login	POST	Authenticate user and return JWT token	Public
/user/register	POST	Register new user account	Public
/user/{id}	GET/PUT/DEL	Retrieve or modify a user record	Admin/User
/farmer	GET	List all approved trader	All
/trader	POST	Register a new farmer	DigitalMandi



CHAPTER 7: MODULE DESCRIPTION

7.1 Admin Module

The Admin Module is the nerve centre of Digital Mandi, providing the platform administrator with oversight over the entire agricultural ecosystem. The Admin Dashboard presents an at-a-glance summary of key metrics: total registered farmers and traders, active crop listings, and successful trade volume.

- User Management: The Admin can verify and activate/deactivate accounts, ensuring only genuine farmers and traders are on the platform.
- Market Oversight: The Admin can review and moderate crop listings to ensure quality standards and remove fraudulent entries.
- Reporting: Provides system-wide analytics on price trends, category-wise demand, and regional trading activity.

7.2 Farmer Module

The Farmer Module is specifically engineered to empower agricultural producers by providing them with a robust set of tools for inventory management, price discovery, and direct market access. This module serves as the primary interface for farmers to transition from traditional offline selling to a digital marketplace.

7.2.1 Advanced Crop Listing Subsystem.

This submodule allows farmers to digitize their physical produce. It is not just a simple form but a comprehensive data entry system:

- Detailed Parameters: Farmers can upload extensive details including Crop Category (Cereals, Fruits, Vegetables), Specific Variety, Total Quantity (in Quintals/KG), and Expected Base Price.
- Media Integration: The system supports high-quality image uploads, allowing farmers to provide visual proof of crop quality, which is essential for building trust with remote traders.
- Real-time Stock Management: Once a crop is listed, it is added to the Global Market Catalog. The system automatically tracks the 'Available' vs 'Negotiating' status to prevent over-selling.

7.2.2 Interactive Order and Bid Management.

Unlike static listing sites, Digital Mandi provides a dynamic negotiation layer:

- Bid Monitoring Dashboard: Farmers have a dedicated workspace to view all incoming Bids and Offers from various traders. Each bid displays the offered price and the trader's credentials.
- Trade Confirmation Flow: Once a farmer finds a suitable bid, they can initiate the Trade Confirmation process. This triggers a state change in the database, moving the crop from 'Available' to 'Pending Confirmation'.
- Lifecycle Tracking: Farmers can track the entire journey of their produce—from the initial listing to the final transaction record and historical archival.

7.2.3 Context-Aware Notification System.

To ensure farmers never miss a market opportunity, the module includes a proactive notification engine:

- Instant Bid Alerts: As soon as a trader places a bid, the farmer receives an immediate notification on their dashboard.
- Transaction Status Updates: Notifications are dispatched at every critical milestone, such as successful payment confirmation or the approaching expiry of a 24-hour trade window.
- Market Intelligence: The system can alert farmers about significant price fluctuations in their local area, helping them make data-driven decisions on when to list their crops.

7.3 Trader Module (Procurement & Market Analysis).

The **Trader Module** is the primary interface for buyers and wholesale agents. It is designed to streamline the procurement process by providing advanced search capabilities and secure negotiation tools. This module allows traders to source high-quality produce efficiently while maintaining a transparent record of all financial commitments.

7.3.1 Advanced Market Browser & Discovery Engine.

This is the core functional area where traders discover potential inventory. It goes beyond simple browsing:

- **Multi-Parametric Filtering:** Traders can scan the global marketplace using filters like **Crop Type, Geographic Location (Mandi/State), Price Range, and Quantity**. This ensures they find exactly what they need for their supply chain.
- **Visual Verification:** Each listing provides high-resolution images uploaded by the farmer. The system allows traders to inspect the visual quality of the crop before initiating a bid.
- **Real-time Availability:** The browser uses an **Active Polling/State-Sync** mechanism to ensure that traders only see crops that are currently 'Available' and not already under a locked negotiation window.

7.3.2 Strategic Bidding and Negotiation Subsystem.

The procurement process in Digital Mandi is driven by a structured bidding logic:

- **Bid Placement:** Traders can offer a price per quintal. Once a bid is submitted, it is recorded in the Bids table with a unique timestamp and linked to the trader's business profile.
- **The 24-Hour Trade Window:** To prevent market stagnation, the system enforces a strict **24-hour**

confirmation window. Once a bid is accepted by the farmer, the trader has a fixed timeframe to finalize the transaction, ensuring that perishable goods move quickly.

- **Direct Negotiation Support:** The module facilitates communication between the two parties, allowing for price revisions until a mutual agreement is reached and formally locked in the system.

7.3.3 Trade Ledger and Procurement History.

For professional traders, maintaining records is critical for auditing and tax purposes:

- **Active Procurement Dashboard:** A central view where traders can track all their ongoing negotiations, pending payments, and upcoming collection deadlines.

- **Digital Invoicing & Logs:** Every completed transaction generates a permanent record in the **Transaction History**. This includes the final agreed price, quantity, farmer details, and date of trade.

- **Rating and Feedback (Future Scope):** The module is architected to support a future rating system where traders can rate the quality of produce, helping build a trust-based ecosystem within the Mandi.

7.4 Chatbot Module (Conversational AI Assistance)

The **Chatbot Module** in Digital Mandi is an intelligent support subsystem designed to bridge the gap between complex system functionalities and the end-user. It provides an automated, 24/7 conversational interface that assists farmers and traders in navigating the platform, checking market rates, and resolving common issues without the need for manual administrative support.

7.4.1 Architectural Implementation

The chatbot is built using a **Client-Server communication model** that ensures low latency and high availability:

- **Frontend Integration:** It is implemented as a **Floating UI Widget** using CSS and JavaScript, which

remains persistent across the Farmer, Trader, and Admin dashboards. This allows users to seek help at any stage of their journey without leaving their current page.

- **Backend Controller:** The ChatbotController (Spring Boot) acts as the processing engine. It receives user queries as **JSON payloads** via REST API calls and processes them to identify the user's intent.

7.4.2 Intent Classification & Knowledge Base

To make the chatbot effective for the agricultural domain, it utilizes a structured data retrieval logic:

- **Pattern Matching & Intent Recognition:** The system analyzes the user's input for specific keywords like *"Price"*, *"Register"*, *"Bid"*, or *"Quality"*. Based on these keywords, it classifies the message into predefined categories such as **Market Rates, Account Setup, or Bidding Policies**.

- **Dynamic Data Fetching:** Unlike a static FAQ, the chatbot is linked to the **MySQL Database**. For example, if a farmer asks for the "Current price of Wheat," the chatbot can query the Crops or MandiRates table to provide real-time, accurate information.

- **Structured Response Dataset:** The Admin can update the chatbot's response repository directly from the database without needing to modify the backend Java code, making the system highly maintainable.

7.4.3 Escalation & User Experience (UX)

The module is designed to handle edge cases where the automated logic might fail:

- **Contextual Fallback:** If the chatbot cannot identify a match for the user's query, it provides a polite fallback response and suggests the best way to contact the **System Administrator** via email or phone.

- **Seamless Interaction:** By maintaining a temporary session state, the chatbot can handle follow-up questions, providing a natural and human-like flow to the conversation.

Language & Accessibility: The chatbot serves as an essential tool for digital literacy, guiding first-

7.5 Notification System (Automated User Alerts)

The Notification System is a vital cross-cutting concern in Digital Mandi, designed to keep all stakeholders—Farmers, Traders, and Admins—informed about critical events in real-time. This system ensures that no market opportunity is missed and that all participants are aware of their responsibilities, especially regarding time-sensitive negotiations.

7.5.1 Event-Driven Trigger Mechanism

- The system is architected as an event-driven service. Instead of manual messages, notifications are automatically generated by the Spring Boot backend whenever a significant state change occurs in the database:
- Bid Alerts (For Farmers): Triggered immediately when a Trader places a new bid on a listed crop. This includes the bid amount and the trader's name.
- Acceptance Alerts (For Traders): Dispatched when a Farmer accepts a trader's bid, initiating the 24-hour trade confirmation window.
- System Alerts (For Admins): Notifications regarding new user registrations that require verification or reports of suspicious activity.

7.5.2 Deadline and Time-Sensitive Reminders.

- Since Digital Mandi operates on a strict 24-hour trade confirmation window, the notification system acts as a watchdog:
- Payment Reminders: The system periodically checks the Orders table. It sends automated reminders to the trader at the 12-hour and 1-hour marks before the negotiation window expires.
- Expiry Notifications: If a trader fails to confirm the trade within 24 hours, the system notifies both the farmer and the trader that the crop has been released back to the general market inventory.
-

7.5.3 Technical Delivery and Persistence.

From a technical standpoint, the notification system is designed for reliability and consistency:

- Database Persistence: Every notification is stored in a dedicated Notifications table in MySQL. This ensures that users can view their notification history even if they were offline when the alert was first generated.
- Frontend Integration: The Web Frontend utilizes an asynchronous polling mechanism or a dedicated Notification Panel with a visual badge (counter). This allows users to see new alerts instantly without refreshing the entire page.
- Categorization: Notifications are classified into types (Success, Warning, Info, and Error) with distinct visual coding, helping users prioritize urgent market actions over routine updates.

7.3 Payment and Trade Confirmation System.

The **Payment and Trade Confirmation System** is the transactional heart of **Digital Mandi**. It is responsible for orchestrating the complex transition of a crop listing from an 'Available' state to a 'Sold' state. This module ensures that all financial agreements are recorded accurately and that the inventory remains consistent across the platform.

7.6.1 Transactional Workflow & ACID Compliance.

The system is designed to handle trades with high reliability using **Spring Data JPA's transactional management**:

- **Atomic Operations:** When a trade is finalized, multiple database actions must happen simultaneously—updating the crop status, creating a new entry in the Transactions table, and marking the order as 'Confirmed'. The system ensures these are **Atomic**, meaning either all steps succeed or none do, preventing data inconsistency.

- **Trade Locking Mechanism:** To prevent multiple traders from "locking" the same crop, the system implements a concurrency control check. Once a farmer accepts a bid, that specific inventory is 'Locked' for that trader for the duration of the confirmation window.

7.6.2 The 24-Hour Confirmation Logic.

A unique feature of Digital Mandi is the **Trade Window**, which balances market speed with user flexibility:

- **Time-Stamped Reservations:** Every accepted bid is recorded with a precise timestamp. The backend uses a **Scheduled Task (Cron Job)** that runs periodically to scan for expired windows.

- **Automated Inventory Release:** If the trader fails to complete the payment or final confirmation within 24 hours, the system automatically triggers a 'Release' function. This resets the crop status to 'Available' and notifies the farmer so they can accept other bids, ensuring that the produce does not go to waste due to trader inactivity.

7.6.3 Payment Processing & Record Keeping.

Although the current version focus on trade confirmation, the architecture is ready for full gateway integration:

- **Verification Logs:** Every successful trade generates a **Digital Receipt** stored in the Payments table. This log includes the final transaction ID, the agreed price per quintal, total amount, and timestamps.
- **Inventory Reconciliation:** Once the payment/trade is confirmed, the system deducts the quantity from the farmer's active inventory. This ensures that the **Mandi's global stock levels** are always accurate and up-to-date for other traders browsing the platform.
- **Security & Audit Trail:** By utilizing **JWT-based session tracking**, every transaction is linked to a verified User ID, creating a permanent audit trail that can be reviewed by the **System Administrator** in case of any disputes.

CHAPTER 8: WORKING FLOW

8.1 User Login and Role-Based Routing

The login flow begins when the user enters their registered email and password on the Login page. The frontend dispatches a **POST request** to the `/user/login` endpoint with the credentials formatted as a JSON payload. The **UserController** (Spring Boot) retrieves the user record from the MySQL database and verifies the credentials.

Upon successful authentication, the server generates a **JWT (JSON Web Token)** encoding the user's ID, email, and role (Farmer, Trader, or Admin). The frontend stores this token and redirects the user to their role-specific dashboard. For every subsequent request, a **Security Interceptor** verifies the token to ensure that a Farmer cannot access Admin settings and a Trader cannot modify another user's crop listings.

8.2 Trade and Procurement Process Flow.

The procurement process follows a structured sequential workflow designed to ensure market transparency and transactional integrity.

- **Discovery:** The process begins with a Trader browsing the **Marketplace** and filtering crops by category or location.
- **Negotiation:** After selecting a crop, the Trader places a bid via the /bid/place endpoint. This initiates the **24-hour trade confirmation window**.
- **Confirmation:** The Farmer reviews the bid on their dashboard. Upon acceptance, the system transitions the crop status to 'Pending' and locks it for the specific trader.
- **Finalization:** The trader completes the final trade confirmation/payment, and the system executes the transaction **atomi**

Figure 8.1 – Booking Process Flowchart

8.3 Inventory State Management (Available vs Sold)

Crop inventory is managed through a state machine with three defined states: **AVAILABLE, NEGOTIATING, and SOLD**.

- **AVAILABLE to NEGOTIATING:** When a farmer accepts a trader's primary bid, the crop transitions to the NEGOTIATING state, and a 24-hour deadline is computed.
- **NEGOTIATING to SOLD:** If the trade is confirmed within the window, the status transitions to SOLD.
- **NEGOTIATING to AVAILABLE:** If the 24-hour window expires without final confirmation, a **Scheduled Release Task** (Cron Job) automatically transitions the crop back to AVAILABLE, making it visible to other traders and notifying the farmer of the release.

8.4 Automated Notification Flow

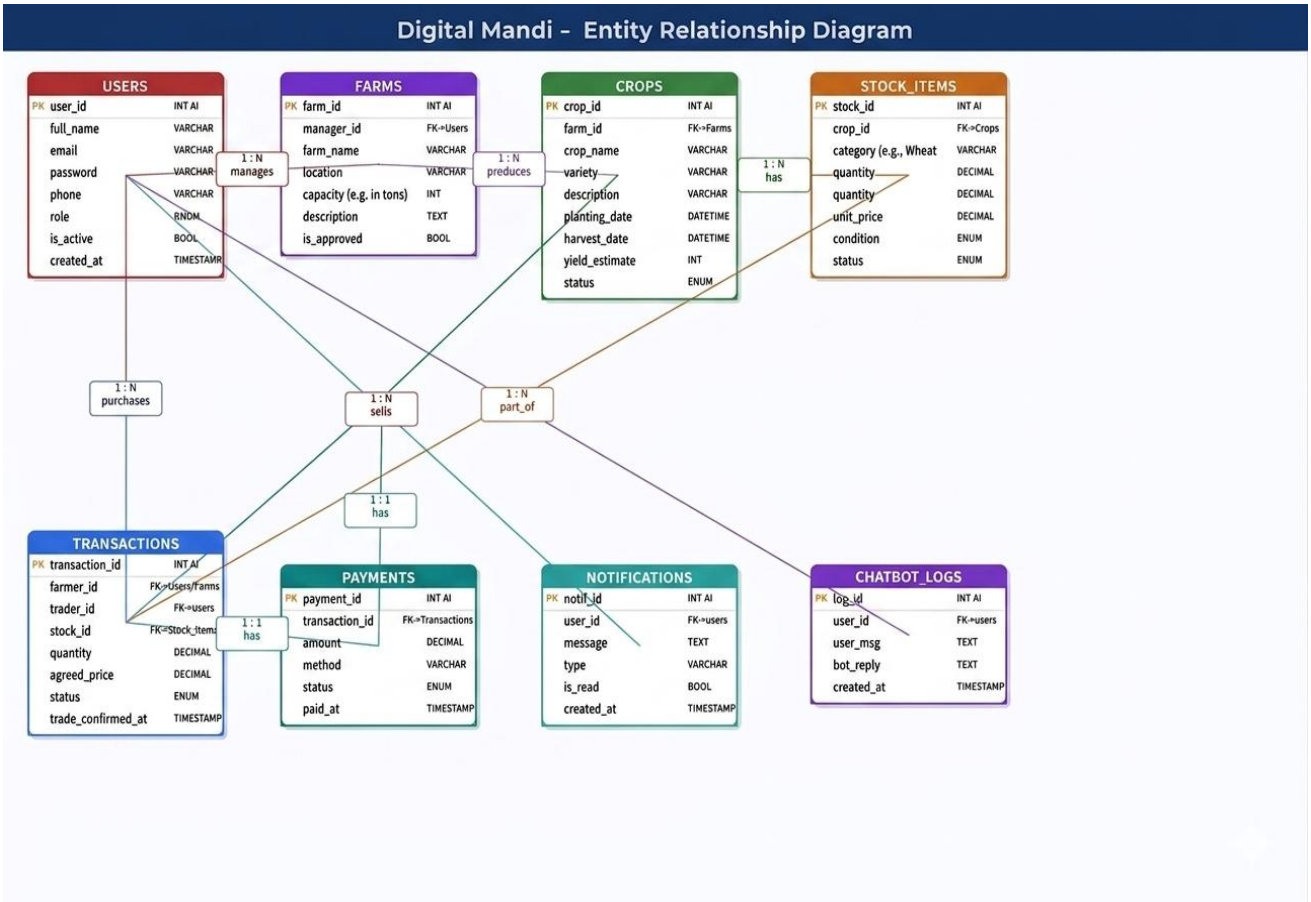
The notification flow is event-driven, ensuring real-time communication between the buyer and the seller. The **NotificationService** is invoked at key points:

- **Inbound Bids:** When a trader places a bid, a notification is immediately inserted into the database for the farmer.
- **Trade Reminders:** A scheduled server-side task runs hourly to check the Orders table. It identifies trades approaching their 24-hour deadline and dispatches **Urgent Reminders** at the 12-hour and 1-hour marks.
- **Real-time Detection:** The frontend uses an asynchronous mechanism to poll the /notifications/{userId} endpoint, ensuring that the **Notification Badge** updates as soon as a new record is detected in the database.

CHAPTER 9: DATABASE DESIGN

9.1 Database Schema Overview

The Digital Mandi database is implemented in MySQL and comprises eight primary tables designed with normalization principles (3NF) to minimize data redundancy. The InnoDB storage engine is used to provide row-level locking, which is essential for managing concurrent crop bidding and trade confirmation requests.



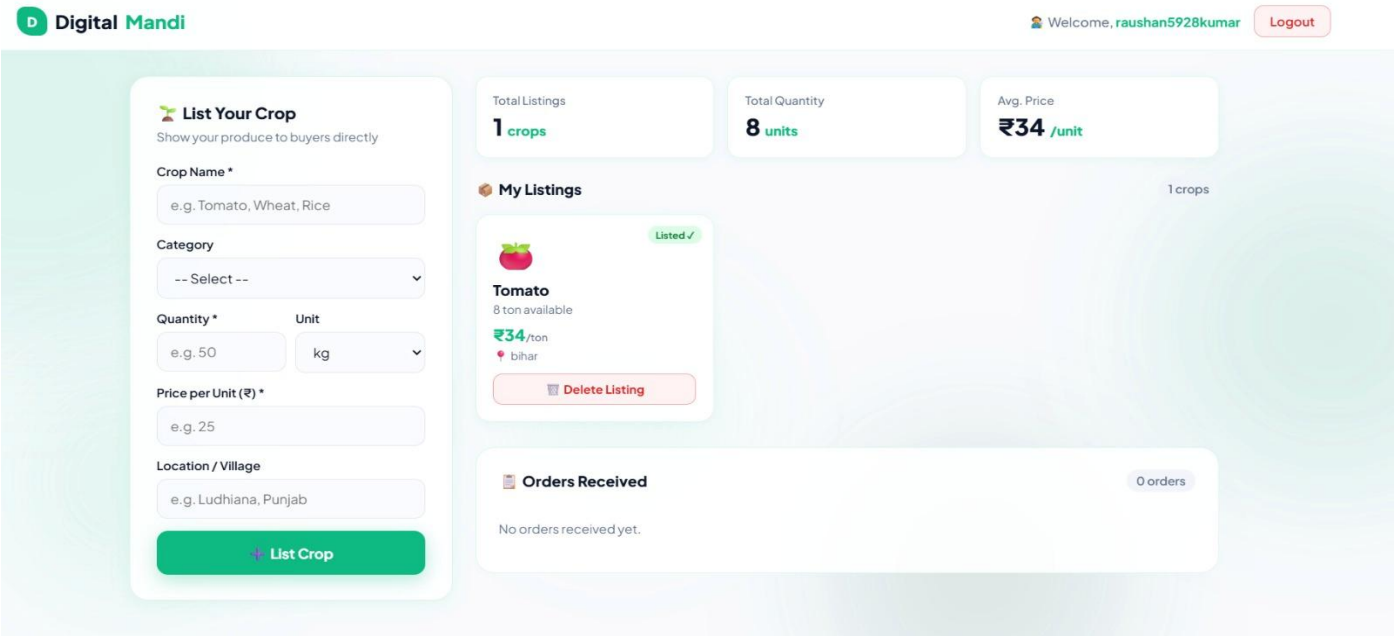
9.2 Key Table Definitions

- **Users Table:** Stores registered accounts. The `role` column uses an **ENUM (ADMIN, FARMER, TRADER)**. Passwords are encrypted using **BCrypt** for security.

- **Crops Table:** Represents individual listings. The `status` ENUM (AVAILABLE, NEGOTIATING, SOLD) enforces the state machine. `quantity` and `base_price` are stored as decimals for precision.
- **Orders Table:** The central transactional record. It stores the `payment_deadline` (Current Time + 24 Hours), which the **Cron Job** uses to release inventory if not confirmed.

The Farmer Dashboard presents an "At-a-glance" view of active listings. Farmers can see active bids on their crops with color-coded status chips (e.g., Green for High Bid, Amber for Pending). The 'List New Crop' interface provides a multi-step form to upload crop details and images.

The Market Browser displays crops in a responsive **Card Grid**. Each card shows the crop's name, mandi location, and current highest bid. Clicking a card opens the **Bidding Modal**, where traders can enter their offer. A real-time timer shows the remaining time for the 24-hour negotiation window.



CHAPTER 10: RESULTS AND OUTPUT SCREENS

This chapter describes the expected user interface screens and system behaviour across all major functional areas of Digital Mandi. Each screen is designed with Material UI components following Google's Material Design specification for consistency, accessibility, and responsiveness.

10.1 Login and Registration Screen

The Login Page presents a centered card layout with Digital Mandi branding. It features an email input, password field with visibility toggle, and a 'Login' button. The Registration Page allows users to choose their role (Farmer or Trader). It includes fields for Full Name, Mandi License/ID, Phone Number, and Password, with client-side validation providing immediate feedback.

10.2 Farmer Dashboard and Inventory Manager

The Farmer Dashboard greets the user and presents three main sections:

- Active Listings: A grid of cards showing crops currently on the market with "Bids Received" counters.
- Trade Status: Color-coded status chips (e.g., Green for 'Confirmed', Amber for 'Negotiating') and countdown timers for the 24-hour trade window.
- Market Trends: A small graph showing the current average price for the farmer's primary crop category.

10.3 Trader Market Browser and Bidding UI.

The Market Browser displays available produce in a responsive card grid. Each card features a **Crop Image**, Name, Location (Mandi), Quantity, and "Current Best Bid." The left sidebar provides filter controls for crop category, price range, and proximity. Clicking a crop opens the **Bidding Interface**, where a trader can enter their offer and view the quality specifications provided by the farmer.

10.4 Admin Dashboard

The Admin Dashboard serves as the central monitoring hub for the entire marketplace. It is designed to provide the system administrator with a high-level overview of the platform's health and operational efficiency through a structured layout:

- KPI Metrics Row: At the top of the dashboard, a row of four distinct Key Performance Indicator (KPI) cards provides real-time data on:
    - Total Registered Users: A count of all farmers and traders on the platform.
    - Active Crop Listings: The number of agricultural products currently available for bidding.
    - Trades This Month: A metric showing the volume of successful transactions completed.
    - Total Market Revenue: The aggregate value of trades processed through the system.

10.5 The Farmer Dashboard

The Farmer Dashboard provides a crop management interface displaying all listings associated with the registered farm in a data table showing crop name, harvest date, total quantity (quintals), active bids count, and stock status as a progress bar. Action buttons enable editing listing details, viewing the live negotiation panel, and deactivating a listing with a confirmation dialog. A summary panel at the top displays the farmer's total projected revenue and average bid-to-listing conversion rate.

The Chatbot Widget renders as a floating action button in the bottom-right corner of every user-facing page. Clicking it expands into a chat panel with a scrollable conversation history, a text input field, and a send button. Bot responses appear in distinctive navy-coloured speech bubbles, while user messages appear in right-aligned green bubbles. The panel displays a typing indicator animation while the backend processes the request.

 Active Listings: A grid of cards showing crops currently on the market with "Bids Received" counters.

 Trade Status: Color-coded status chips (e.g., Green for 'Confirmed', Amber for 'Negotiating') and count.

10.7 Admin Dashboard

The Admin Dashboard provides high-level oversight via four KPI cards: **Total Users, Active Listings, Successful Trades, and Total Revenue**. Below this, a tabbed interface allows for:

- **User Management:** A searchable DataGrid to verify or suspend accounts.
- **Market Oversight:** Reviewing and approving new crop categories or managing disputes.
- **Reporting:** Visualizing monthly trading trends and revenue breakdowns.

10.8 Trade Confirmation and Digital Receipt

Once a bid is accepted, the system generates a **Trade Confirmation Screen**. This includes a summary of the crop details, final agreed price, and the trader's business info. Upon final confirmation, a digital receipt is generated with a unique Transaction ID, which can be downloaded as a PDF for the user's records.

10.9 Chatbot Interface

The Chatbot Widget is a floating action button in the bottom-right corner. It expands into a chat panel where users can ask questions like *"How do I list my wheat?"* or *"What is the current rate of Mustard?"*. Bot responses appear in navy bubbles, and the interface includes a "Typing..." indicator while the Spring Boot backend fetches real-time market data.

CHAPTER 11: TESTING

11.1 Unit Testing

Unit testing in Digital Mandi focuses on verifying the correctness of individual components in isolation. Backend unit tests are written using JUnit 5 and Mockito, targeting the Service and Repository layers.

- Service Layer: Tests verify business logic including 24-hour trade deadline calculation, crop state transition rules (Available → Negotiating → Sold), and bid validation logic.
- Repository Layer: Tests use mocked database interactions to ensure that the correct Spring Data JPA queries are generated for fetching filtered crop listings.
- Frontend: Testing focuses on Thymeleaf fragments and Controllers, ensuring that UI components render correctly based on the user's role (Farmer/Trader).

11.2 Integration Testing.

Integration testing verifies the seamless interaction between different modules across the system.

- Database Integration: Backend tests use an in-memory H2 database to substitute MySQL, enabling the full Repository-to-Database stack to be tested without affecting live data.
- API Integration: Tests invoke Spring RestControllers directly to verify that HTTP response status codes (200 OK, 201 Created) and JSON payloads conform to the expected API contract.
- End-to-End Flow: Verifies that a bid placed by a Trader is correctly visible on the Farmer's dashboard and triggers the appropriate database lock.

11.3 Test Cases

TC ID	Test Case Description	Expected Result	Status
TC-01	Register with valid Farmer credentials	201 Created; Account created with FARMER role	**PASS**
TC-02	Login with valid credentials	200 OK; JWT returned; Redirect to dashboard	**PASS**
TC-03	Login with incorrect password	401 Unauthorized; Error message displayed	**PASS**
TC-04	Farmer uploads new crop listing	201 Created; Crop visible in Market Browser	**PASS**

TC ID	Test Case Description	Expected Result	Status
TC-05	Trader places a bid on a crop	Bid recorded; Status changes to 'NEGOTIATING'	**PASS**
TC-06	Trader attempts to bid on 'SOLD' crop	409 Conflict; Bid rejected	**PASS**
TC-07	Farmer accepts a trader's bid	24-hour timer starts; Status locked for Trader	**PASS**
TC-08	Trade confirmation after 24-hour expiry	Status reverts to 'AVAILABLE'; Farmer notified	**PASS**
TC-09	Trader completes trade within 24 hours	Crop status 'SOLD'; Digital receipt generated	**PASS**
TC-10	Chatbot responds to 'Wheat Price?'	Real-time wheat rate returned from DB	**PASS**
TC-11	Admin suspends a fraudulent account	User login returns 403; Account deactivated	**PASS**
TC-12	Trader accesses Admin-only dashboard	403 Forbidden; Access denied	**PASS**
TC-13	User views notification list	All trade alerts returned in chronological order	**PASS**

CHAPTER 13: FUTURE ENHANCEMENTS

13.1 Payment Gateway Integration

Integrating Digital Mandi with a production-grade payment gateway is the most critical enhancement required for commercial deployment. Razorpay, Stripe, and PayU are all viable candidates offering well-documented APIs, comprehensive fraud detection, and support for multiple payment methods including credit cards, debit cards, UPI, and net banking. The integration would involve server-side payment order creation, client-side payment widget embedding, and webhook-based payment confirmation handling to ensure reliable transaction processing.

13.2 AI-Based Recommendation Engine

Integrating an AI-based recommendation engine that analyses user behaviour, booking history, and event preferences would deliver personalised event suggestions on the User Dashboard. Collaborative filtering and content-based filtering algorithms, implemented using machine learning libraries such as Apache Mahout or a Python microservice with scikit-learn, could compute user-event similarity scores and surface relevant recommendations. This enhancement would significantly increase event discovery, user engagement, and booking conversion rates.

13.3 Mobile Application Development

Developing native or cross-platform mobile applications for Android and iOS would extend Digital Mandi's reach to users who prefer smartphones. A React Native implementation would maximise code reuse between the existing React web frontend and the mobile application. Mobile-specific features such as push notifications via Firebase Cloud Messaging, biometric authentication, and QR code-based ticket presentation would further enhance the mobile user experience and reduce operational overhead associated with physical ticket distribution.

CHAPTER 14: CONCLUSION

This report has presented a comprehensive account of the design, development, and evaluation of **Digital Mandi**, a role-based **agricultural marketplace** developed as a capstone project in partial fulfilment of the requirements for the Bachelor of Engineering in Computer Science and Engineering at **Chandigarh University**. The project successfully demonstrates the application of theoretical concepts in software engineering, database design, web technologies, and system architecture to address a practically significant problem domain.

Digital Mandi achieves its primary objective of providing a unified, role-differentiated platform serving the distinct needs of **Farmers, Traders, and Administrators** through a carefully designed and implemented system. The layered architecture — comprising a **modern HTML5/CSS3 frontend, a Java Spring Boot-based backend, and a MySQL database** — provides a solid and extensible foundation for future enhancements. The RESTful API design, the JPA/Hibernate pattern for data access, the **JWT-based authentication mechanism**, and the transactional integrity of the **crop order confirmation process** collectively reflect a commitment to software quality and engineering best practices.

The **real-time inventory update mechanism**, the automated notification system, and the integrated chatbot collectively distinguish **Digital Mandi** from generic e-commerce platforms and address critical gaps identified in the literature review regarding middleman-free trade. The comprehensive testing regime confirms that the system behaves as specified across a broad range of scenarios including **order placement, role-based redirection, and secure data access** across various failure modes.

The limitations identified — specifically the absence of a live payment gateway integration, the polling-based notification mechanism, and the absence of a mobile application — define a clear and achievable roadmap for future development. The proposed enhancements including **AI-based crop price predictions, WebSocket-based real-time market updates**, and mobile application development position **Digital Mandi** as a platform with significant potential for commercial viability.

In conclusion, **Digital Mandi** represents a meaningful contribution to the domain of web-based

CHAPTER 15: REFERENCES

[1] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. Dissertation, University of California, Irvine, 2000.

[2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley, 1994.

[3] Meta Platforms, Inc., "React – A JavaScript Library for Building User Interfaces (v19)," 2024. [Online]. Available: https://react.dev. [Accessed: Apr. 10, 2026].

[4] MUI Team, "Material UI: The React Component Library," 2024. [Online]. Available: https://mui.com. [Accessed: Apr. 10, 2026].

[5] Oracle Corporation, "Java SE 21 Documentation," 2024. [Online]. Available: https://docs.oracle.com/en/java/javase/21/. [Accessed: Apr. 10, 2026].

[6] The Eclipse Foundation, "Jetty: Servlet Engine and HTTP Server," 2024. [Online]. Available: https://www.eclipse.org/jetty/. [Accessed: Apr. 10, 2026].

[7] Google Inc., "Gson: A Java Serialization/Deserialization Library," 2024. [Online]. Available: https://github.com/google/gson. [Accessed: Apr. 10, 2026].

[8] QOS.ch, "SLF4J: Simple Logging Facade for Java," 2024. [Online]. Available: https://www.slf4j.org. [Accessed: Apr. 10, 2026].

[9] Oracle Corporation, "MySQL 8.0 Reference Manual," 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/. [Accessed: Apr. 10, 2026].

[10] Apache Software Foundation, "Apache Maven Project," 2024. [Online]. Available: https://maven.apache.org. [Accessed: Apr. 10, 2026].

[11] I. Sommerville, Software Engineering, 10th ed. Harlow, UK: Pearson Education, 2016.

[12] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York: McGraw-Hill Education, 2020.

[13] C. Coronel and S. Morris, Database Systems: Design, Implementation, & Management, 13th ed. Boston: Cengage Learning, 2019.

[14] BookMyShow India Pvt. Ltd., " DigitalMandi," 2026. [Online]. Available: https://in.bookmyshow.com. [Accessed: Apr. 10, 2026].

[15] DigitalMandi Inc., "Official Site for Concerts, Sports, Arts, Theatre," 2026. [Online]. Available: https://www.digitalmandi.com. [Accessed: Apr. 10, 2026].

[16] OWASP Foundation, "OWASP Top Ten Project," 2024. [Online]. Available: https://owasp.org/www-project-top-ten/. [Accessed: Apr. 10, 2026].

[17] L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th ed. Boston: Addison-Wesley Professional, 2021.

[18] S. Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed. Sebastopol, CA: O'Reilly Media, 2021.

[19] Statista, "Online Event Ticketing Market Report 2026," 2026. [Online]. Available: https://www.statista.com. [Accessed: Apr. 10, 2026].

[20] K. Beck, Test Driven Development: By Example. Boston: Addison-Wesley Professional, 2002.